brevity penalty. In our experiments, we set $N = 4$ and use average weight.

4.2 Baseline and Ablation Test Settings

We compare our proposed method with three baseline models, namely:

- **GCG** (Zou et al., 2023): An discrete optimization method of adversarial suffixes based on gradient to induce model output of harmful content.
- **AutoDan[Liu]** (Liu et al., 2023): Using a carefully designed hierarchical genetic algorithm on the basis of GCG to enhance the concealment of jailbreak prompts;
- **AutoDan[Zhu]** (Zhu et al., 2023): Guided by the dual goals of jailbreak and readability, optimize from left to right to generate readable jailbreak prompts that bypass perplexity filters;
- **GPTFuzzer** (Wichers et al., 2024): Using templates written by humans as initial seeds, then automating mutations to generate new templates.

We performed an ablation study to validate the necessity of each component in our proposed ASETF framework. Specifically, we compared ASETF to three modified frameworks lacking key modules of our full system. The brief introduction of these methods are as follows:

- **ET-suffix**: In the process of fine-tuning the translation model, only the suffix is translated without considering the context;
- **ET-ce**: When optimizing the continuous embedding vector ϕ in Section 3.1, only use L_{ce} without L_{mmd};
- **ET-origin**: In the process of fine-tuning the translation model, do not add noise ϵ to the embedding vector of suffix E_s;
- **Rand-suffix**: Randomly extract tokens from a vocabulary as attack suffixes.

4.3 Main Result

4.3.1 Ad-hoc LLM attack with ad-hoc suffix

In this section, we optimize each harmful instruction on a single to-be-attacked model to obtain adversarial suffixes, and use an embedded translation model targeting that attack model to transform the obtained suffixes as Figure 2(a). The Table 1 shows our experimental results.

To-Be-Attacked Model	Method	Perplexity ↓	ASR ↑		Time(s) ↓	Self-BLEU ↓
			ASR_{prefix}	ASR_{gpt}		
LLaMA 2	GCG	1513.09±1193.03	0.90	0.61	233.87±227.51	0.698
	AutoDan[Liu]	51.76±37.65	0.88	0.67	347.43±158.21	0.431
	AutoDan[Zhu]	39.17±25.71	0.84	0.63	262.14±235.40	0.469
	GPTFuzzer	61.63±41.15	0.81	0.45	-	0.728
	ASETF	**32.59±19.38**	**0.91**	**0.74**	**104.53±73.58**	**0.399**
VICUNA LLM	GCG	1214.34±992.52	0.93	0.71	142.63±131.62	0.728
	AutoDan[Liu]	53.88±24.19	0.90	0.76	309.65±147.55	**0.387**
	AutoDan[Zhu]	44.09±26.28	0.91	0.75	204.81±193.17	0.494
	GPTFuzzer	61.63±41.15	0.71	0.62	-	0.728
	ASETF	**43.02±20.09**	**0.94**	**0.82**	**94.26±33.80**	0.417
Mistral AI	GCG	1598.31±1322.49	**0.95**	0.70	234.17±236.79	0.661
	AutoDan[Liu]	51.17±33.72	0.91	0.73	382.07±257.64	0.428
	AutoDan[Zhu]	42.19±33.85	0.92	0.75	301.26±196.50	**0.425**
	GPTFuzzer	61.63±41.15	0.77	0.58	-	0.728
	ASETF	**39.98±32.31**	**0.95**	**0.80**	**95.32±63.29**	0.441
Alpaca with Safe-RLHF	GCG	1338.08±1362.19	0.89	0.73	295.48±200.98	0.596
	AutoDan[Liu]	48.29±32.21	0.86	0.75	371.59±282.14	0.478
	AutoDan[Zhu]	43.68±37.36	**0.90**	0.76	304.57±217.03	
	GPTFuzzer	61.63±41.15	0.73	0.58	-	0.728
	ASETF	**38.75±37.28**	**0.90**	**0.81**	**92.18±68.55**	**0.436**

Table 1: The result of our method and baseline method in Ad-hoc LLM attack with ad-hoc suffix. ↓ means the lower the better, while ↑ means to higher the better. (Note that the perplexity of "GCG" are extremely high since their generated prompts are unreadable dummy text.)

The experimental results show that compared with traditional gradient based discrete optimization suffix or methods based on jailbreak templates, our method has a higher attack success rate and improves the fluency of input prompts. Crucially, our method has higher computational efficiency due to optimization in continuous embedding spaces.

Due to the contextual information incorporated during the training process, our method produces adversarial suffixes and instructions that are more semantically relevant, enhancing the robustness of adversarial samples. As shown in Table 2, the experimental results indicate that even when paraphrasing prompts as defense, the success rate of our method still higher than other methods.

To-Be-Attacked Model	Method	ASR_{gpt} ↑	
		$Before-Para$	$After-Para$
LLaMA 2	GCG	0.61	0.21
	AutoDan[Liu]	0.67	0.19
	AutoDan[Zhu]	0.63	0.21
	ASETF	**0.74**	**0.37**
VICUNA LLM	GCG	0.71	0.32
	AutoDan[Liu]	0.65	0.33
	AutoDan[Zhu]	0.60	0.29
	ASETF	**0.75**	**0.48**

Table 2: The result of our method and baseline method in Ad-hoc LLM attack before/after paraphrasing. We use ChatGPT to paraphrasing the generated adversarial prompt, Before-para indicating before paraphrasing and After-para indicating after paraphrasing.

4.3.2 Ad-hoc LLM attack with universal suffix

We use the method in Section 3.1 to optimize the adversarial suffix for 25 harmful instructions simultaneously, in order to obtain the same suffix that

can generalize all harmful instructions.

To-Be-Attacked Model	Method	Perplexity ↓	ASR ↑		Time(s) ↓
			ASR_{prefix}	ASR_{gpt}	
LLaMA 2	GCG	1513.09±1193.03	**0.88**	0.61	965.75±881.08
	AutoDan[Liu]	41.81±34.14	0.78	0.50	1139.21±992.02
	AutoDan[Zhu]	43.44±47.50	0.81	0.52	859.10±974.53
	ASETF	**37.90±33.27**	**0.88**	**0.67**	**427.52±419.36**
VICUNA LLM	GCG	1214.34±992.52	0.90	0.71	895.78±953.55
	AutoDan[Liu]	47.50±35.57	0.83	0.65	940.61±863.96
	AutoDan[Zhu]	49.26±43.87	0.88	0.60	905.90±798.67
	ASETF	**40.31±36.08**	**0.92**	**0.75**	**469.31±403.13**

Table 3: The result of our method and baseline method in Ad-hoc LLM attack with universal suffix

The experimental results in Table 3show that our method achieves state-of-the-art attack success rate and also improves the fluency of universal adversarial suffixes. More importantly, it significantly reduces the time required to obtain universal adversarial suffixes.

4.3.3 Transferable LLM attack with ad-hoc suffix

Training on multiple models simultaneously is a common approach to improve the transferability of adversarial samples. For each harmful instruction, we trained adversarial suffixes both the Llama2-7b-chat model and Vicuna-7b-v1.5, and transferred the obtained adversarial suffixes to other LLMs. We chose three LLMs, Vicuna-13b, Llama2-13b chat, and Chatglm3-6b, to test the transferability of the adversarial suffixes we obtained. Due to the direct transfer of adversarial suffixes, both Perplexity and Self-BLEU values are the same when attack different LLMs. The specific experimental results are in Table 4:

Method	To-Be-Attacked Model	Perplexity ↓	ASR		Self-BLEU ↓
			ASR_{prefix} ↑	ASR_{gpt} ↑	
ASETF	Vicuna-13b	32.17±19.41	**0.64**	**0.59**	0.451
	Llama2-13b		**0.46**	**0.32**	
	ChatGLM3-6b		**0.54**	**0.39**	
GCG	Vicuna-13b	1870.73±1084.43	0.47	0.36	0.623
	Llama2-13b		0.26	0.17	
	ChatGLM3-6b		0.39	0.28	

Table 4: The results of our method and GCG on the transferability of adversarial suffixes

The experimental results indicate that the adversarial suffixes obtained by our method have a certain degree of transferability. Compared with other method based on adversarial suffixes, ASETF has a higher success rate of transfer attacks, but compared to the direct attack method using model gradient information, the success rate of transfer attacks has significantly decreased. This may due to the significant differences between different LLMs in the pre-train stage.

4.4 Ablation Test

We conducted ablation experiments using the above methods described in 4.2

To-Be-Attacked Model	Method	Perplexity ↓	ASR ↑		Self-BLEU ↓
			ASR_{prefix}	ASR_{gpt}	
LLaMA 2	ET-suffix	73.07±52.51	0.85	0.73	0.583
	ET-ce	87.82±61.09	0.69	0.57	0.559
	ET-origin	49.22±47.95	0.76	0.69	0.549
	Rand-suffix	1126.55±1346.92	0.27	0.13	**0.355**
	ASETF	**32.59±19.38**	**0.91**	**0.74**	0.399
VICUNA LLM	ET-suffix	63.74±49.67	0.90	0.79	0.552
	ET-ce	71.96±53.05	0.81	0.65	0.531
	ET-origin	44.01±42.51	0.71	0.57	0.581
	Rand-suffix	1126.55±1346.92	0.31	0.22	**0.355**
	ASETF	**43.02±20.09**	**0.94**	**0.82**	0.417

Table 5: Ablation results of attacking Llama2-7b-chat and Vicuna-7b-v1.5 models

The results of ablation tests in Table 5 indicate that removing the MMD loss during the optimization process of continuously embedded vectors ϕ, or removing contextual information within the embedding translation framework, significantly reduces the fluency of adversarial samples. Additionally, removing the random noise ϵ added during the training process of the translation model also leads to a decrease in the performance of our method. Furthermore, randomly selected tokens as suffixes fail to jailbreak attacks, demonstrating the need for carefully designed adversarial suffixes.

5 Conclusion

In this paper, we propose a robust and comprehensive framework for generating semantically rich and coherent adversarial inputs. Initially, we derive an embedding translation model by undertaking the task of text reconstruction from embeddings on raw text. Subsequently, input the vector trained in continuous embedding space into the translation model, resulting in adversarial suffixes. Through experimentation on multiple Large Language Models (LLMs), our method significantly reduces computational costs compared to optimizing suffixes in discrete space, while achieving a higher attack success rate and improving the fluency and diversity of the suffix. This contributes to the formulation of more effective defense strategies and in our approach, the process of obtaining the embeddings for adversarial suffixes and the training of the translation model are decoupled, implying that our method is plug-and-play. This method is expected to be further applied in text adversarial attacks beyond just LLM jailbreak attacks.

Limitations

Firstly, from the experimental results, it is discernible that universal adversarial suffixes, optimized for multiple instructions, exhibit a lower success rate in attacks compared to independent adversarial suffixes. This phenomenon could be attributed to the necessity for universal adversarial suffixes to encapsulate a broader spectrum of information. However, the capacity for information representation of discrete tokens depends on their length, and an extended length implies a more complex optimization process.

Upon further examination of cases, we observe that if the adversarial suffixes generated by the translation model are biased towards semantics related to harmful instructions in the preceding text, the attack is prone to failure. Conversely, if they lean towards maintaining the consistency of embeddings, it can lead to textual incoherence. Our method does not explicitly model these two objectives separately; hence, it is not possible to artificially control which target the generated adversarial suffixes are more inclined towards.

Ethics Statement

Firstly, the goal of this article is to promote the exploration of defense mechanisms for Large Language Models (LLMs), rather than to obtain illegal content from LLMs, as outlined in the appendix. Secondly, the training data used in this article are all public data, and there is no data falsification in the experimental results. Our code will be submitted with the paper and uploaded to GitHub.

References

Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90.

Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2023. Safe rlhf: Safe reinforcement learning from human feedback. *arXiv preprint arXiv:2310.12773*.

Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2023. Masterkey: Automated jailbreak across multiple large language model chatbots. *arXiv preprint arXiv:2307.08715*.

David Glukhov, Ilia Shumailov, Yarin Gal, Nicolas Papernot, and Vardan Papyan. 2023. Llm censorship: A machine learning challenge or a computer security problem? *arXiv preprint arXiv:2307.10719*.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*.

Alec Helbling, Mansi Phute, Matthew Hull, and Duen Horng Chau. 2023. Llm self defense: By self examination, llms know they are being tricked. *arXiv preprint arXiv:2308.07308*.

Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. 2021. Unsolved problems in ml safety. *arXiv preprint arXiv:2109.13916*.

Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*.

Aounon Kumar, Chirag Agarwal, Suraj Srinivas, Soheil Feizi, and Hima Lakkaraju. 2023. Certifying llm safety against adversarial prompting. *arXiv preprint arXiv:2309.02705*.

Linyi Li, Tao Xie, and Bo Li. 2023. Sok: Certified robustness for deep neural networks. In *2023 IEEE symposium on security and privacy (SP)*, pages 1289–1310. IEEE.

Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. 2022. Diffusion-lm improves controllable text generation. *Advances in Neural Information Processing Systems*, 35:4328–4343.

Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 61–68.

Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2023. Autodan: Generating stealthy jailbreak prompts on aligned large language models. *arXiv preprint arXiv:2310.04451*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506.